

LakeCat

covers lakecat (0.1.0)

An Engine-Close Catalog Foundation for QueryGraph

Alexy Khrabrov

chiefscientist.org

&

Codex with ChatGPT 5.5

Contents

1 LakeCat	1
1.1 Preface	1
1.2 What A Catalog Is	2
1.3 What Iceberg Does	2
1.4 The Current Catalog State	3
1.5 What Intermediation Loses	4
1.6 Why Sail Matters	4
1.7 Why Move Catalog Work Closer To The Engine	5
1.8 LakeCat's Thin Boundary	5
1.9 The Read Path	6
1.10 The Commit Path	6
1.11 The Durable Spine	7
1.12 Grust For Graph Concepts	7
1.13 TypeSec For Security	8
1.14 Rust-First Engines And The V3 To V4 Path	8
1.15 OSI, OpenLineage, And Responsible Semantic Handoff	9
1.16 QueryGraph.ai When LakeCat Is Done	10
1.17 Implementation Shape	11
1.18 Standard Compatibility And Extensions	11
1.19 Operating The Book's Example System	12
1.20 What Comes Next	12

1 LakeCat

1.1 Preface

LakeCat is a Rust-native Iceberg catalog foundation for QueryGraph. It starts from a deliberately conservative claim: the ordinary Iceberg REST catalog boundary must keep working for ordinary engines, but the next catalog also needs to become a governed control plane for Rust-first planning, semantic graph handoff, lineage, and agent access.

In this repository the catalog is LakeCat and the engine-side lakehouse implementation is Sail. The idea is not to invent a new table format or to make every client learn a new protocol. It is to keep Iceberg compatibility at the boundary while moving the parts that need deep Iceberg knowledge closer to the Rust engine that can reason about them.

This book starts from first principles. It explains what a catalog is, why Apache Iceberg puts so much responsibility into metadata, what Sail already does with Iceberg metadata, and why a passive catalog is not enough for governed agentic systems. Then it shows the LakeCat shape: a thin Rust catalog that owns identity, tenancy, metadata-pointer state, policy gates, idempotent commits, and integration events, while delegating reusable engine, graph, and security work to Sail, Grust, and TypeSec.

The intended end state is QueryGraph.ai. LakeCat should become the catalog foundation for the next QueryGraph: a place where standard engines still see an Iceberg REST catalog, while QueryGraph can bootstrap Croissant, CDIF, OSI, ODRL, OpenLineage, TypeSec security receipts, and a Grust-backed graph from the same governed source of truth.

1.2 What A Catalog Is

A data catalog is often described as a place that lists datasets. That is true, but too small. A real catalog is the control plane between names, storage, metadata, identity, and intent.

At minimum, a catalog answers four questions:

1. What table does this name mean?
2. Where is its current metadata?
3. Who is allowed to read, write, plan, or administer it?
4. What changed, when, and under whose authority?

In a traditional database, the catalog is embedded inside the database system. The engine owns the table definitions, statistics, indexes, permissions, and transaction log. A client asks the database a question and the same system resolves names, checks permissions, plans the query, and executes it.

A lakehouse splits that system apart. Data files live in object storage. Metadata files live beside them. Multiple engines may read and write the same tables. A catalog becomes the agreement point: it maps a logical table name to the current metadata pointer and arbitrates updates to that pointer.

That pointer is deceptively important. If the catalog points at metadata version 17, the table is version 17. If a writer prepares version 18 and wins the compare-and-swap update, the table becomes version 18. If it loses, the table does not partially change. The catalog is not the table format, but it is the place where table history becomes visible and durable.

For human-scale analytics this can sound like bookkeeping. For agentic systems it becomes a trust boundary. A catalog can know the principal, the warehouse, the namespace, the table, the current snapshot, the requested columns, the row restriction, the storage profile, and the policy receipt. If that information is captured before planning and committed after state changes, the catalog becomes the control plane for governed data access rather than a passive address book.

1.3 What Iceberg Does

Apache Iceberg is a table format for large analytic tables. Its core design is simple: put the table's truth in explicit metadata files, and let engines use that metadata to plan reads and validate writes without relying on directory listing or fragile storage conventions.

An Iceberg table has a current metadata file. That metadata names schemas, partition specs, sort orders, snapshots, properties, and the current snapshot. Snapshots point to manifest lists. Manifest lists point to manifests. Manifests describe data files and delete files. The data files normally use formats such as Parquet, Avro, or ORC.

This layered metadata gives Iceberg its practical power:

- Schema evolution is explicit.
- Partition evolution is explicit.
- Snapshot isolation is explicit.
- Commit conflicts can be checked before the current pointer advances.
- Scan planning can prune manifests and files before touching data.
- Deletes can be represented without rewriting every data file immediately.
- Multiple engines can interoperate because the table state is stored in a shared, documented format.

The catalog role in Iceberg is intentionally narrow. Standard clients need to load table metadata, create namespaces and tables, commit changes, and sometimes receive credentials or scan tasks. The catalog must not require business semantics or proprietary metadata for normal table access. If standard clients have to call a non-standard endpoint to read an ordinary table, compatibility is already broken.

LakeCat preserves that rule. Iceberg metadata stays pristine. Business semantics, policy, graph, lineage, and agent state are derived control-plane or graph data. The table remains an Iceberg table.

1.4 The Current Catalog State

The current lakehouse catalog market is converging on a simple fact: the catalog is no longer a sidecar. It is the place where table identity, tenancy, commits, credentials, governance, and interoperability are negotiated.

Hive Metastore gave early lakehouses a familiar namespace and table registry, but it was born for a different format era. Cloud warehouses built proprietary catalogs around their own engines. Nessie explored Git-like table references and branching. Unity Catalog turned governance and sharing into a central product surface. Lakekeeper has shown how a modern open Iceberg REST catalog can model warehouses, projects, credentials, soft deletion, and management APIs without polluting table metadata. Polaris has made the strongest standards-centered move: an open Iceberg REST catalog with vendor gravity, clear governance ambition, and a credible route for many engines to meet at the same catalog boundary.

That is why Polaris is a winner. It is not because it is the last word in catalog architecture. It is winning because it occupies the obvious shared surface at the right moment:

- It speaks Iceberg REST instead of asking every engine to learn a new table protocol.
- It treats the catalog as a security and governance surface, not merely a pointer map.
- It gives enterprises an open center of gravity around Iceberg while the warehouse, query-engine, and cloud-storage layers remain plural.
- It can be adopted incrementally: engines can keep reading tables, while operators gain a real control plane.

LakeCat should learn from that. The winning move is not to reject Polaris-style compatibility. The winning move is to keep that compatibility and then ask what a Rust-native, QueryGraph-facing

catalog can do when it is allowed to plan near Sail, emit graph through Grust, and ask TypeSec for authorization proofs.

1.5 What Intermediation Loses

Catalogs win by sitting between engines and data. That same position can also flatten the system.

When the catalog is only an intermediary, it often sees the table name, the current metadata pointer, the caller, and perhaps a credential request. The engine sees the schema, manifests, statistics, deletes, partition evolution, scan filters, and physical plan. The governance system sees policy. The lineage system sees an event after the fact. The semantic layer sees a separate model. Each system receives a shard of the truth.

The loss is not merely elegance. It is operational:

- Policy can be checked before access but not carried into scan planning.
- Credentials can be vended without proving why a raw credential exception was allowed.
- Lineage can describe that something happened but not bind to the exact governed plan, snapshot, policy, and table metadata.
- Semantic layers can drift from the physical tables they describe.
- Agents can receive broad storage access when they should have received a governed plan and short-lived, narrow task set.
- Engines can optimize with file statistics while catalogs remain blind to the consequences of those choices.

The catalog should not become a replacement engine. But it should not be a blind intermediary either. LakeCat's thesis is that the catalog can stay thin and standard while still becoming engine-close at the moments where correctness, security, and semantics depend on planning evidence.

1.6 Why Sail Matters

Sail is the Rust lakehouse engine path. It already contains the pieces that should understand Iceberg deeply: catalog abstractions, generated Iceberg REST models, DataFusion table providers, manifest pruning, metadata-as-data paths, write and commit plumbing, and format-version checks.

That matters because scan planning and commit validation are not generic catalog chores. They require knowledge of Iceberg schemas, projections, partition specs, sort orders, snapshots, manifests, data files, delete files, statistics, and expression models. If LakeCat reimplements those details, it becomes a second Iceberg engine. The same bug class appears twice, and the catalog begins to drift away from the planner that actually executes work.

LakeCat takes the opposite path. Put reusable Iceberg and planning behavior in Sail. Let LakeCat call Sail for the parts that require engine-grade knowledge. Keep LakeCat responsible for the catalog boundary: identity, tenancy, durable state, policy gates, idempotency, audit, outbox, and standard REST compatibility.

The current LakeCat architecture follows this line. The service exposes an Iceberg REST-compatible surface under `/catalog/v1`. The Sail-facing engine path handles scan planning, table-status

conversion, metadata preparation, manifest expansion, and standard response validation. LakeCat keeps only the additional context it must own: which principal asked, which policy narrowed the read, which metadata pointer was current, which commit won, and which event should be replayed to graph and lineage sinks.

1.7 Why Move Catalog Work Closer To The Engine

A passive catalog returns metadata locations and lets the client plan. That is compatible, but it is not enough for the next generation of governed systems. Agents, notebooks, services, and model pipelines need stronger guarantees:

- A policy should be enforced before a scan is planned.
- Column restrictions should narrow the projection before file tasks are produced.
- Row predicates derived from policy should become mandatory scan filters.
- A stateless `fetchScanTasks` call should not widen a prior governed plan.
- Credentials should be short-lived, scoped, and audited.
- Commit retries should be idempotent and should not replay a different request.
- Graph and lineage side effects should reflect committed catalog state, not best-effort handler side effects.

Those guarantees sit between catalog state and engine planning. If the catalog is too far from the engine, it can check a policy and then hand the client a metadata pointer, hoping the client preserves the restriction. If the engine is too far from the catalog, it can plan efficiently but may not know the governance receipt or durable identity context.

LakeCat fuses those two moments. LakeCat authorizes the request, derives the effective restriction, and asks Sail to plan or validate against the current metadata pointer. Sail performs the Iceberg work. LakeCat returns the standard shape and records the governance evidence.

The result is still an Iceberg catalog. The difference is that governed reads and writes are planned through the same Rust implementation that QueryGraph will use downstream.

1.8 LakeCat's Thin Boundary

LakeCat should be thin, but thin does not mean trivial. It owns the durable catalog state that must be correct even when external sinks are unavailable.

The core LakeCat responsibilities are:

- Serve the Iceberg REST Catalog API for standard clients.
- Model projects, warehouses, namespaces, tables, views, and storage profiles.
- Persist metadata pointers and compare-and-swap commit history.
- Validate idempotency keys and replay only matching commit bodies.
- Resolve request identity from headers, bearer tokens, agents, and TypeDID envelopes.
- Ask TypeSec for authorization decisions and persist receipts.
- Route scan planning and commit preparation through Sail.
- Record audit and outbox events inside the catalog transaction.
- Drain committed events to Grust and OpenLineage sinks.

- Publish a QueryGraph bootstrap bundle.

The deliberately excluded responsibilities are just as important:

- LakeCat does not invent a table format.
- LakeCat does not fork Iceberg manifest pruning.
- LakeCat does not own graph traversal or graph query behavior.
- LakeCat does not own security semantics or agent trust semantics.
- LakeCat does not author QueryGraph’s final business semantic model.

That boundary gives each sibling project a clear job. Sail owns reusable Iceberg and planning behavior. Grust owns graph schema, storage, and query mechanics. TypeSec owns policy, capabilities, TypeDID envelopes, secure agents, and authorization semantics. QueryGraph owns the semantic application built on top.

1.9 The Read Path

The LakeCat read path begins like a standard catalog request and ends with a governed Sail plan.

1. A client asks to load or plan a table through the Iceberg REST surface.
2. LakeCat resolves the warehouse, namespace, table, and current metadata pointer.
3. LakeCat resolves the request identity.
4. LakeCat asks TypeSec whether the principal can perform the requested action.
5. TypeSec returns a decision and any enforced restrictions.
6. LakeCat turns those restrictions into a ReadRestriction: allowed columns, required row predicate, purpose, policy hash, and audit context.
7. LakeCat asks Sail to plan against the current metadata pointer with the effective projection and filters.
8. Sail validates Iceberg expressions against generated REST models and table schema, expands manifests, applies conservative file-bound pruning when metrics are present, and carries delete-file references into file scan tasks.
9. LakeCat returns Iceberg-compatible plan and task responses.
10. LakeCat records audit and outbox events that can later be projected into graph and lineage.

The important detail is that the policy restriction becomes part of planning, not a note beside it. An empty client projection under a column restriction means the allowed columns. A client projection can narrow further, but cannot widen. During `fetchScanTasks`, LakeCat recomputes the current restriction and requires the token to satisfy it. A stale or legacy token cannot silently expand back to all columns.

1.10 The Commit Path

The write path follows the same principle: LakeCat owns the catalog transaction, Sail owns reusable Iceberg validation and metadata preparation.

1. A client sends an Iceberg commit request, optionally with an idempotency key.
2. LakeCat validates the request shape and the idempotency key.
3. LakeCat resolves identity and asks TypeSec for the commit capability.

4. LakeCat loads the current metadata pointer from the store.
5. LakeCat delegates Iceberg update validation and metadata assembly to Sail.
6. LakeCat writes the new metadata object through the warehouse storage profile.
7. LakeCat advances the table pointer with compare-and-swap.
8. LakeCat persists idempotency, audit, pointer-log, and outbox records.
9. If the store rejects the commit after a local metadata write, LakeCat cleans up the uncommitted metadata object when it can do so safely.
10. Outbox draining projects the committed event to graph and lineage sinks.

Idempotency is part of correctness. Reusing the same key for the same commit can return the stored response. Reusing the same key for a different body must conflict. LakeCat persists a normalized request hash and stores only audit-safe evidence, not raw secrets or raw idempotency keys.

1.11 The Durable Spine

LakeCat's durable local spine uses the Rust `turso` crate behind the `turso-local` feature. The store contract remains portable, but the local foundation is not SQLx. The important tables are not an application afterthought; they define the catalog's control-plane memory:

- projects and warehouses;
- storage profiles;
- namespaces and tables;
- metadata pointer log;
- idempotency records;
- soft deletes;
- policy bindings;
- audit events;
- outbox events.

Object storage remains the source of Iceberg metadata files. Turso stores the atomic pointer, management state, idempotency evidence, and event record. This mirrors the Iceberg catalog contract: metadata files describe the table; catalog state decides which metadata file is current.

1.12 Grust For Graph Concepts

Catalog events naturally form a graph. A warehouse contains namespaces. A namespace contains tables. A table has columns, snapshots, manifests, files, policies, commits, scan plans, principals, and lineage runs. QueryGraph needs that graph, but LakeCat should not become a graph database.

Grust owns the graph layer. It is the place for reusable graph taxonomy, projection builders, graph stores, traversal indexes, Cypher support, and typed or untyped graph operations. LakeCat's responsibility is narrower: translate committed catalog events into a bounded envelope and pass it through a catalog-facing sink.

In practice this means LakeCat can emit graph events for stable catalog facts:

```
Warehouse CONTAINS Namespace
Namespace CONTAINS Table
```


Table HAS_COLUMN Column
Table GOVERNED_BY Policy
Principal CAN_PLAN ScanPlan
Commit DERIVED_FROM Snapshot
LineageRun USED_BY QueryGraphModel

High-cardinality file and manifest facts should stay queryable through Iceberg/Sail metadata-as-data unless Grust provides a reusable taxonomy and storage strategy for them. The graph should be powerful, but the catalog must not smuggle a second lakehouse engine into its event sink.

The current local direction already proves the boundary: LakeCat's `grust-local` sink calls Grust-owned LakeCat projection helpers, and the Grust Cypher boundary test verifies catalog graph projection without making LakeCat own Cypher parsing, traversal, or graph execution.

1.13 TypeSec For Security

LakeCat is a policy enforcement point, not the author of security semantics. TypeSec owns the semantics: RBAC, ODRL-style policy composition, typed capabilities, TypeDID envelopes, secure agents, credential issuance decisions, and authorization proofs.

Every externally meaningful action should pass through TypeSec:

- catalog configuration reads;
- namespace creation and listing;
- table creation, load, scan planning, commit, drop, and restore;
- credential vending;
- policy management;
- graph and lineage reads.

LakeCat gathers the request context and asks TypeSec for a decision. The context can include principal DID, agent DID, bearer-derived subject, warehouse, namespace, table, columns, snapshot, requested credential duration, purpose, and active policy bindings. TypeSec returns a decision and receipt. LakeCat persists the receipt with audit-safe hashes and applies the resulting restrictions before Sail plans.

This is where ODRL becomes operational. An ODRL-style policy can say that a principal may read only certain columns, only for a purpose, or only with an enforced row predicate. LakeCat parses the minimal enforceable subset it needs to narrow the plan, but policy composition and authorization semantics belong to TypeSec. LakeCat should ask TypeSec, not grow a parallel security language.

Credential vending follows the same rule. Raw credential vending is an audited exception. Governed Sail-planned reads are the default path for agents and untrusted principals. When credentials must be issued, TypeSec checks the `credentials.issue` capability for the exact secret reference and LakeCat returns only scoped, short-lived credential configuration.

1.14 Rust-First Engines And The V3 To V4 Path

The new Rust-first engine path matters because it changes where catalog intelligence can live. Sail is not a Java service with Rust bindings bolted on the side. It is a Rust engine path built around

Arrow, DataFusion, generated Iceberg REST models, catalog provider traits, manifest pruning, metadata-as-data scans, and table-status conversion.

That shape gives LakeCat a better option than reimplementation. LakeCat can ask Sail for typed Iceberg behavior instead of parsing just enough JSON to survive a request. That matters for Iceberg v3 and the emerging v4 work. Format v3 already pushes catalogs and engines toward richer metadata, row lineage, deletion semantics, and better interoperability around advanced table state. Format v4 is still settling, but it is plainly moving the lakehouse toward more adaptive and structured metadata rather than less.

LakeCat should evolve under three rules:

1. Conform to Iceberg v3 for ordinary clients.
2. Preserve unknown and emerging v4 metadata without claiming settled semantics.
3. Prefer typed Sail support as soon as Sail exposes it, using JSON passthrough only as a compatibility bridge.

That gives LakeCat room to support v4-ready capability flags, round-trip tests, metadata extension preservation, and future metadata-tree planning without forking Iceberg. The catalog can become more intelligent while the table remains portable. The standard path stays boring. The governed Sail path becomes richer.

1.15 OSI, OpenLineage, And Responsible Semantic Handoff

QueryGraph needs more than physical table access. It needs a semantic picture: datasets, fields, policies, lineage, graph relationships, and anchors that can survive import. LakeCat should publish that picture without pretending to be QueryGraph.

The LakeCat bootstrap bundle contains:

- Semantic Croissant and CDIF projections for dataset and field discovery.
- An OSI handoff with stable dataset and field anchors.
- ODRL policy artifacts and TypeSec policy context.
- OpenLineage events for catalog changes, scan plans, commits, and maintenance.
- A Grust-ready graph envelope.
- A manifest that hashes each emitted artifact.

The OSI boundary is deliberately careful. LakeCat should not author rich business metrics, dimensions, joins, ontology claims, or authoritative semantic names. It can publish stable anchors and governed source metadata. QueryGraph owns the final semantic model.

This is the Responsible Semantic Layer boundary. Semantic Croissant and CDIF make datasets and fields discoverable and exchangeable. OSI gives QueryGraph a stable handoff for semantic anchors without forcing LakeCat to own business meaning. OpenLineage records how catalog and planning events happened. Together they let the semantic layer be responsible because it can be traced back to catalog state, policy, and lineage, not just to a hand-authored model file.

OpenLineage fits the catalog outbox. A committed table create, scan plan, commit, soft delete, restore, or maintenance action can become a lineage event. Because the event is drained from

a durable outbox after the catalog transaction, lineage reflects committed state rather than a handler's best-effort side effect.

1.16 QueryGraph.ai When LakeCat Is Done

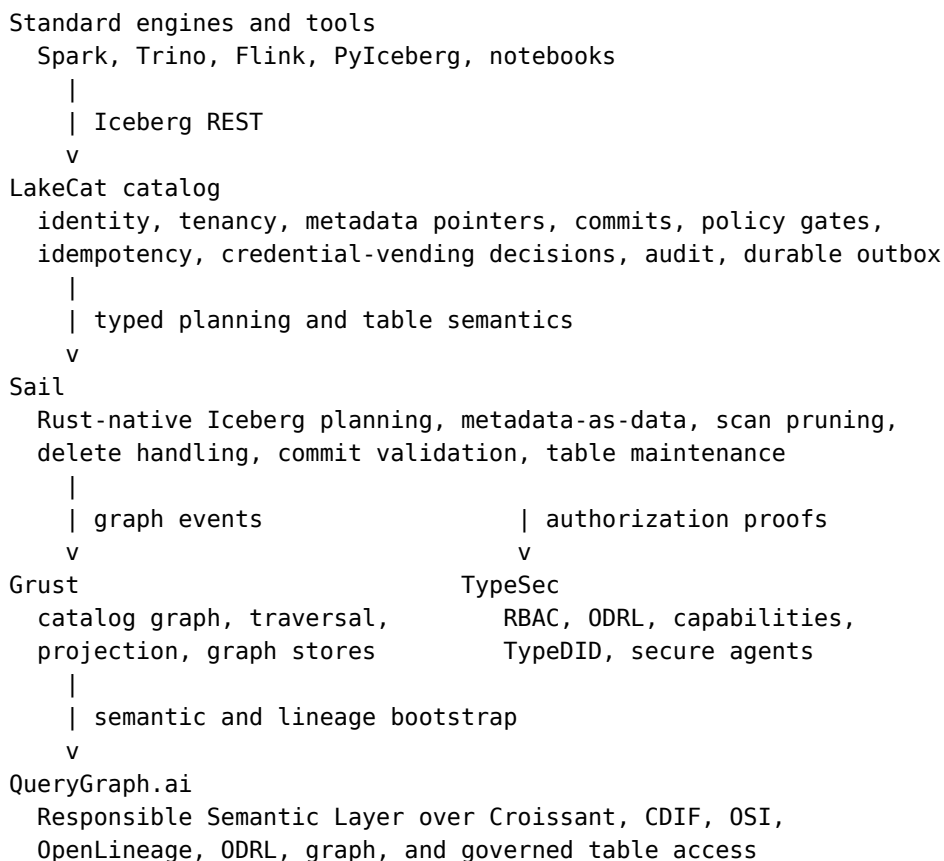
QueryGraph.ai is the enterprise lakehouse this work is pointing toward. QueryGraph needs the catalog to be more than a storage address book. It wants to answer questions over data, metadata, semantics, policy, and lineage as one governed graph. That requires a catalog foundation that can speak to ordinary Iceberg clients and also publish trustworthy control-plane facts.

LakeCat supports that foundation by exposing a QueryGraph bootstrap endpoint:

```
/querygraph/v1/bootstrap
```

The bundle gives QueryGraph an import contract. QueryGraph can verify hashes, load catalog graph envelopes through Grust, inspect policy artifacts through TypeSec, and attach semantic modeling work to stable dataset and field anchors. The import path should prove that LakeCat is the substrate, not a standalone demo.

When LakeCat is done, the QueryGraph.ai architecture looks like this:



Basic catalog use remains optional and standard. A normal Iceberg engine can load and commit tables without knowing QueryGraph exists. The enhanced path is there for enterprises that want more: governed Sail-planned reads, TypeSec authorization receipts, ODRL rights, TypeDID agent

identity, OpenLineage replay, Semantic Croissant/CDIF publication, OSI handoff, and Grust graph loading.

That is the core motivation for LakeCat. The next QueryGraph.ai should not bolt governance, graph, and lineage onto tables after the fact. It should begin from a catalog that already records governed state transitions and exposes standard, engine-close planning.

1.17 Implementation Shape

The current workspace shape expresses the architecture directly:

```
crates/
  lakecat-core      stable IDs, errors, time, config, content hashes
  lakecat-api       Iceberg REST request/response adapters
  lakecat-store     catalog state traits and Turso-backed implementation
  lakecat-sail      Sail provider bridge and privileged planning client
  lakecat-graph     catalog-facing Grust sink/adapters
  lakecat-security  TypeSec integration and authorization receipts
  lakecat-lineage   OpenLineage projection and event receipts
  lakecat-querygraph Croissant/CDIF/OSI/ODRL/OpenLineage bootstrap projection
  lakecat-service   axum service, middleware, auth, routing
  lakecat-cli       admin, local demo, conformance, bootstrap export
```

Feature gates keep integrations honest:

```
sail-local    use local Sail APIs for planning and provider integration
typesec-local use local TypeSec APIs for governance and TypeDID verification
grust-local   use local Grust APIs for catalog graph projection
turso-local   use the Turso-backed durable store
```

Embedded defaults stay safe for tests. Real integrations are explicit. That matters because LakeCat is a foundation, not a pile of optional demos. A test that only uses memory stores should not accidentally depend on a sibling repo. A test that claims to validate TypeSec should enable `typesec-local` and call TypeSec.

1.18 Standard Compatibility And Extensions

LakeCat must be boring where standards require boring behavior. Standard Iceberg clients should be able to use the catalog without knowing QueryGraph exists. Business semantics and agent state must not be required custom Iceberg metadata.

Extensions belong beside the standard path:

- `/catalog/v1` serves Iceberg REST compatibility.
- management APIs handle warehouses, policies, profiles, and operational state.
- `/querygraph/v1/bootstrap` publishes QueryGraph import artifacts.
- feature-gated Sail paths provide governed scan planning and local provider integration.

Format v4 work should follow the same rule. Prefer typed Sail support when available. JSON passthrough can bridge compatibility, but it is not the desired long-term implementation. Round-

trip tests should prove LakeCat preserves unknown or evolving metadata without claiming settled semantics too early.

1.19 Operating The Book's Example System

The local development posture is intentionally small:

```
cargo run -p lakecat-cli -- config
cargo run -p lakecat-cli -- storage-profile-list
cargo run -p lakecat-cli -- policy-list
cargo run -p lakecat-cli -- qglake-fixture \
  --output target/qglake/lakecat-bootstrap.json
cargo run -p lakecat-cli -- bootstrap-export \
  --output lakecat-bootstrap.json
```

The important thing is what these commands exercise. They are not a separate product surface. They touch the same catalog, policy, bootstrap, and QueryGraph export contracts that the service uses.

1.20 What Comes Next

LakeCat is a direction more than a single release. The next slices should keep making the architecture more true:

1. Remove temporary Sail patch bridges when the required helpers are available upstream.
2. Keep pushing reusable Iceberg table-status, planning, metadata-as-data, and commit helpers into Sail.
3. Make the transactional outbox the only path for graph and lineage side effects.
4. Expand TypeSec-backed capability checks until every privileged path is unbypassable.
5. Keep graph taxonomy and Cypher behavior in Grust.
6. Keep OSI as a QueryGraph-owned semantic handoff rather than a LakeCat-authored business model.
7. Prove the bootstrap bundle through QueryGraph import on every meaningful public-surface change.

The payoff is a catalog foundation that feels ordinary to Iceberg clients and rich to QueryGraph. Tables remain portable. Policies become enforceable. Graph and lineage become replayable. Sail plans close to the data. QueryGraph gets a trustworthy substrate for the next version of its lakehouse intelligence.